

Приложение А

Исходный текст программы

Исходный текст приложения WebApi

UserController.cs – контроллер пользователей

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Update.Internal;
using translog_APIшка.Model;

namespace translog_APIшка.Controllers;

[ApiController]
[Route("api/[controller]")]
public class UserController : ControllerBase
{
    [HttpGet("login")]
    public IActionResult Login(string login, string password)
    {
        var db = new TransLogCourseContext();
        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }

        var users = db.Users.FromSqlRaw(
            $"select * from users where username = '{login}' and password = '{password}'").ToList();
        if (users.Count() == 0)
        {
            return Unauthorized(new { message = "Неверный логин или пароль!" });
        }
        else
        {
            foreach (var user in users)
            {
                return Ok(new
                {
                    message = "Вход выполнен",
                    userId = user.UserId,
                    login = user.Username,
                    password = user.Password,
                    roleId = user.RoleId,
                    fullname = user.FullName,
                    numberhone = user.Numberphone,
                    isActive = user.IsActive
                });
            }
        }

        return Unauthorized(new { message = "Ошибка авторизации" });
    }

    [HttpPost("register")]
    public IActionResult Register(RegisterModel model)
    {

```

Продолжение приложения А

```
if (!ModelState.IsValid)
    return BadRequest(ModelState);
var db = new TransLogCourseContext();
var users = db.Users.FromSqlRaw($"select * from users where username = '{model.Username}'").ToList();
if (users.Count() != 0)
{
    return BadRequest(new { message = "Такой логин уже есть!" });
}

var newUser = new User
{
    Username = model.Username,
    Password = model.Password,
    RoleId = model.RoleId ?? 0,
    FullName = model.fullname,
    Numberphone = model.numberphone,
    IsActive = model.isActive
};
db.Users.Add(newUser);
db.SaveChanges();
return Ok(new
{
    message = "Регистрация прошла успешно!",
    userId = newUser.UserId,
    login = newUser.Username,
    password = newUser.Password,
    roleId = newUser.RoleId,
    fullname = newUser.FullName,
    numberphone = newUser.Numberphone,
    isActive = newUser.IsActive
});
}

[HttpGet("GetUser")]
public IActionResult GetUser()
{
    var db = new TransLogCourseContext();
    if (!ModelState.IsValid)
        return BadRequest(ModelState);
    var users = db.Users.ToList();
    if (users.Count() == 0)
    {
        return BadRequest(new { message = "пользователей нету!" });
    }
    else
    {
        return Ok(users);
    }

    return Unauthorized(new { message = "Ошибка" });
}

[HttpDelete("DeleteUser")]
public IActionResult DeleteUser(int userId)
{

```

Продолжение приложения А

```
var db = new TransLogCourseContext();

var user = db.Users.FirstOrDefault(u => u.UserId == userId);

if (user == null)
    return BadRequest(new { message = "Пользователь не найден!" });

var orders = db.Orders.Where(o => o.UserId == userId).ToList();
db.Orders.RemoveRange(orders);

var drivers = db.Drivers.Where(d => d.UserId == userId).ToList();
db.Drivers.RemoveRange(drivers);

db.Users.Remove(user);

db.SaveChanges();

return Ok(new { message = "Пользователь удалён" });
}
[HttpPut("UpdateUser")]
public IActionResult updateUser(UpdateUserModel model)
{
    var db = new TransLogCourseContext();
    var user = db.Users.FirstOrDefault(u => u.UserId == model.UserId);
    if (user == null)
        return BadRequest(new { message = "Пользователь не найден!" });

    user.FullName = model.FullName;
    user.Username = model.Username;
    user.Numberphone = model.Numberphone;
    user.RoleId = model.RoleId;

    db.SaveChanges();

    return Ok(new { message = "Пользователь обновлён!" });
}
}
public class RegisterModel
{
    public string Username { get; set; } = null!;
    public string Password { get; set; } = null!;
    public int? RoleId { get; set; }
    public string fullname { get; set; }
    public string numberphone { get; set; }
    public int isActive { get; set; }
}
public class UpdateUserModel
{
    public int UserId { get; set; }
    public string FullName { get; set; }
    public string Username { get; set; }
    public string Numberphone { get; set; }
    public int RoleId { get; set; }
}
```

DriverController.cs – контроллер водителей

Продолжение приложения А

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using translog_APIшка.Model;

namespace translog_APIшка.Controllers;

[ApiController]
[Route("api/[controller]")]
public class DriverController : ControllerBase
{
    [HttpGet("GetDrivers")]
    public IActionResult GetDrivers()
    {
        var db = new TransLogCourseContext();
        var drivers = db.Drivers.Include(d => d.User).Include(d => d.Vehicle).ThenInclude(v =>
v.VehicleType).ToList();
        if (drivers.Count == 0)
            return NotFound(new { message = "Водителей нету" });
        return Ok(drivers);
    }

    [HttpPost("AddDriver")]
    public IActionResult AddDriver(DriverModel model)
    {
        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }
        var db = new TransLogCourseContext();
        var newDriver = new Driver
        {
            UserId = model.UserId,
            DriverId = model.DriverId,
            LicenseCategories = model.LicenseCategories,
            VehicleId = model.VehicleId
        };
        db.Drivers.Add(newDriver);
        db.SaveChanges();
        return Ok(new
        {
            message = "Водитель добавлен!",
            newDriver
        });
    }
}

public class DriverModel
{
    public int DriverId { get; set; }

    public int UserId { get; set; }

    public string? LicenseCategories { get; set; }

    public int? VehicleId { get; set; }
```

```
}
```

OrderController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using translog_APIшка.Model;
using translog_APIшка.Services;
using Order = translog_APIшка.Model.Order;
```

```
namespace translog_APIшка.Controllers;
```

```
[ApiController]
[Route("api/[controller]")]
public class OrderController : ControllerBase
{
    [HttpGet("GetOrder")]
    public IActionResult GetOrder()
    {
        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }
        var db = new TransLogCourseContext();
        var orders = db.Orders
            .Include(o => o.Vehicle)
            .ThenInclude(v => v.Drivers)
            .ThenInclude(d => d.User)
            .Include(o => o.User)
            .ToList();
        if (orders.Count() == 0)
        {
            return Unauthorized("Заказов нету");
        }
        else
        {
            return Ok(orders);
        }
    }

    [HttpPost("CreateOrder")]
    public IActionResult CreateOrder([FromBody] OrderModel model)
    {
        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }
        var db = new TransLogCourseContext();
        var newOrder = new Order
        {
            OrderId = model.OrderId,
            ReceivedAt = model.ReceivedAt,
            Status = model.Status,
            Weight = model.Weight,
            VolumeM3 = model.VolumeM3,
            DeparturePoint = model.DeparturePoint,
```

Продолжение приложения А

```
        ArrivalPoint = model.ArrivalPoint,
        Description = model.Description,
        UserId = model.UserId,
        VehicleId = model.VehicleId,
        DepartureTime = model.DepartureTime,
        ArrivalTime = model.ArrivalTime,
    };
    db.Orders.Add(newOrder);
    db.SaveChanges();
    return Ok(new
    {
        message = "Заказ добавлен!",
        newOrder
    });
}

[HttpPut("UpdateOrder")]
public IActionResult UpdateStatus(int OrderId, [FromBody] OrderModel model)
{
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }
    var db = new TransLogCourseContext();
    var order = db.Orders.FromSqlRaw($"select * from orders where order_id = '{OrderId}'").ToList();
    if (order.Count == 0)
    {
        return Unauthorized("Такого заказа нет!");
    }
    else
    {
        foreach (var o in order)
        {
            o.Status = model.Status;
            if (model.DepartureTime != null) o.DepartureTime = model.DepartureTime;
            if (model.ArrivalTime != null) o.ArrivalTime = model.ArrivalTime;
            if (model.VehicleId != null) o.VehicleId = model.VehicleId;
            if (model.Distance_km != null) o.DistanceKm = model.Distance_km;
            if (model.Price != null) o.Price = model.Price;
        }
        db.SaveChanges();
        return Ok(new { message = "Заказ обновлен!", order });
    }
}

[HttpGet("GetHistory")]
public IActionResult GetHistory(int Userid)
{
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }
    var db = new TransLogCourseContext();
    var order = db.Orders.FromSqlRaw($"select * from orders where user_id = '{Userid}'").ToList();
    if (order.Count == 0)
    {
        return Unauthorized("Истории нет!");
    }
}
```

Продолжение приложения А

```
    }
    else
    {
        return Ok(order);
    }
}
[HttpGet("GetReceipt")]
public IActionResult GetReceipt(int orderId)
{
    var db = new TransLogCourseContext();

    var order = db.Orders.FirstOrDefault(o => o.OrderId == orderId);
    if (order == null) return NotFound();

    var pdfService = new pdfService();
    var pdfBytes = pdfService.GenerateReceiptPdf(order);

    return File(pdfBytes, "application/pdf", $"check_{orderId}.pdf");
}

[HttpGet("GetNakladnya")]
public IActionResult GetNakladnya(int orderId)
{
    var db = new TransLogCourseContext();

    var order = db.Orders
        .Include(o => o.Vehicle)
        .ThenInclude(v => v.Drivers)
        .ThenInclude(d => d.User)
        .FirstOrDefault(o => o.OrderId == orderId);

    if (order == null)
        return NotFound();

    var pdfService = new PdfNakladnaya();

    var pdfBytes = pdfService.GenerateNakladnayaPdf(order);

    return File(
        pdfBytes,
        "application/pdf",
        $"nakladnaya_{orderId}.pdf"
    );
}

}

public class OrderModel
{
    public int OrderId { get; set; }

    public DateTime? ReceivedAt { get; set; }

    public int UserId { get; set; }

    public int? VehicleId { get; set; }
```

Продолжение приложения А

```
public string Status { get; set; } = null!;  
  
public decimal? Weight { get; set; }  
  
public decimal? VolumeM3 { get; set; }  
  
public string? DeparturePoint { get; set; }  
  
public string? ArrivalPoint { get; set; }  
  
public DateOnly? DepartureTime { get; set; }  
  
public DateOnly? ArrivalTime { get; set; }  
  
public string? Description { get; set; }  
public int? Distance_km { get; set; }  
public int? Price { get; set; }  
}
```

RoleController.cs – контроллер для ролей

```
using Microsoft.AspNetCore.Mvc;  
using translog_APIшка.Model;  
  
namespace translog_APIшка.Controllers;  
  
[ApiController]  
[Route("api/[controller]")]  
public class RoleController : ControllerBase  
{  
    [HttpGet("getRole")]  
    public IActionResult GetRole()  
    {  
        var db = new TransLogCourseContext();  
        var roles = db.Roles.ToList();  
        return Ok(roles);  
    }  
}
```

VehicleController.cs – контроллер для транспортных средств

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
using translog_APIшка.Model;  
  
namespace translog_APIшка.Controllers;  
  
[ApiController]  
[Route("api/[controller]")]  
public class VehicleController : ControllerBase  
{  
    [HttpGet("GetTransport")]  
    public IActionResult GetTransport()  
    {  
        if (!ModelState.IsValid)  
        {  
            return BadRequest(ModelState);  
        }  
    }  
}
```


Продолжение приложения А

```
var db = new TransLogCourseContext();
var vehicle = db.Vehicles
    .Include(v => v.Drivers).ThenInclude(v => v.User)
    .Include(v => v.VehicleType)
    .ToList();
if (vehicle.Count == 0)
{
    return Unauthorized("Машин нету");
}
else
{
    return Ok(vehicle);
}
}

[HttpPost("AddTransport")]
public IActionResult AddTransport(VehicleModel model)
{
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }

    var db = new TransLogCourseContext();
    var newVehicle = new Vehicle
    {
        LicensePlate = model.LicensePlate,
        Brand = model.Brand,
        Model = model.Model,
        PayloadKg = model.PayloadKg,
        VolumeM3 = model.VolumeM3,
        VehicleTypeId = model.VehicleTypeId,
    };
    db.Vehicles.Add(newVehicle);
    db.SaveChanges();
    if (model.UserId != null)
    {
        var driver = db.Drivers.FirstOrDefault(d => d.UserId == model.UserId);
        if (driver != null)
        {
            driver.VehicleId = newVehicle.VehicleId;
        }
        else
        {
            db.Drivers.Add(new Driver
            {
                UserId = model.UserId.Value,
                VehicleId = newVehicle.VehicleId,
            });
        }

        db.SaveChanges();
    }

    return Ok(new
    {

```

Продолжение приложения А

```
        message = "Машина добавлена!",
        newVehicle
    });
}

[HttpPut("UpdateTransport")]
public IActionResult UpdateTransport(int vehicleId, VehicleModel model)
{
    var db = new TransLogCourseContext();
    var vehicle = db.Vehicles
        .Include(v => v.Drivers)
        .FirstOrDefault(v => v.VehicleId == vehicleId);

    if (vehicle == null)
        return NotFound("Такой машины нету");

    vehicle.LicensePlate = model.LicensePlate;
    vehicle.Brand = model.Brand;
    vehicle.Model = model.Model;
    vehicle.PayloadKg = model.PayloadKg;
    vehicle.VolumeM3 = model.VolumeM3;
    vehicle.VehicleTypeId = model.VehicleTypeId;

    if (model.UserId != null)
    {
        var driver = db.Drivers.FirstOrDefault(d => d.UserId == model.UserId);
        if (driver != null)
            driver.VehicleId = vehicleId;
        else
            db.Drivers.Add(new Driver { UserId = model.UserId.Value, VehicleId = vehicleId });
    }
    else
    {
        var currentDriver = db.Drivers.FirstOrDefault(d => d.VehicleId == vehicleId);
        if (currentDriver != null)
            currentDriver.VehicleId = null;
    }

    db.SaveChanges();
    return Ok(new { message = "Машина обновлена!" });
}

[HttpDelete("DeleteTransport")]
public IActionResult DeleteTransport(int vehicleId)
{
    var db = new TransLogCourseContext();
    var vehicle = db.Vehicles.FirstOrDefault(v => v.VehicleId == vehicleId);
    if (vehicle == null)
        return NotFound(new { message = "Машина не найдена!" });

    var drivers = db.Drivers.Where(d => d.VehicleId == vehicleId).ToList();
    foreach (var d in drivers)
        d.VehicleId = null;

    var orders = db.Orders.Where(o => o.VehicleId == vehicleId).ToList();
    foreach (var o in orders)
        o.VehicleId = null;
}
```

Продолжение приложения А

```
        db.SaveChanges();
        db.Vehicles.Remove(vehicle);
        db.SaveChanges();
        return Ok(new { message = "Машина удалена!" });
    }
}
```

```
public class VehicleModel
{
    public int VehicleId { get; set; }

    public string? LicensePlate { get; set; }

    public string? Brand { get; set; }

    public string? Model { get; set; }

    public decimal? PayloadKg { get; set; }

    public decimal? VolumeM3 { get; set; }
    public int? VehicleTypeId { get; set; }

    public int? UserId { get; set; }
}
```

VehicleTypeController.cs – контроллер для типов транспортного средства

```
using Microsoft.AspNetCore.Mvc;
using translog_APIшка.Model;

namespace translog_APIшка.Controllers;

[ApiController]
[Route("api/[controller]")]
public class VehicleTypeController : ControllerBase
{
    [HttpGet("GetTypes")]
    public IActionResult GetTypes()
    {
        var db = new TransLogCourseContext();
        var types = db.VehicleTypes.ToList();
        return Ok(types);
    }
    [HttpPost("AddType")]
    public IActionResult AddType([FromBody] VehicleTypeModel model)
    {
        var db = new TransLogCourseContext();
        var newType = new VehicleType
        {
            Name = model.Name,
            Description = model.Description,
            PricePerKm = model.PricePerKm
        };
        db.VehicleTypes.Add(newType);
        db.SaveChanges();
        return Ok(newType);
    }
}
```

Продолжение приложения А

```
[HttpPut("UpdateType")]
public IActionResult UpdateType([FromBody] VehicleTypeModel model)
{
    var db = new TransLogCourseContext();
    var type = db.VehicleTypes.FirstOrDefault(t => t.TypeId == model.TypeId);
    if (type == null) return NotFound();
    type.PricePerKm = model.PricePerKm;
    db.SaveChanges();
    return Ok(type);
}
}
public class VehicleTypeModel
{
    public int TypeId { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
    public decimal PricePerKm { get; set; }
}
```

Исходный текст веб-приложения

Authorization.css – стили для авторизации

```
* { margin: 0; padding: 0; box-sizing: border-box; }

#app, html, body { height: 100%; background-color: #FFFAFA; }

.auth { display: flex; justify-content: center; align-items: center; min-height: 100vh; width: 100%; }

#background { display: flex; flex-direction: column; align-items: center; width: 100%; gap: 10px; background-color: white; }

img { width: 400px; height: 250px; }

#translog, #log { font-family: Inter; font-weight: bold; font-size: 32px; color: #1D2D50; }

#log { color: #1A5FBB; }

.label { display: flex; justify-content: center; align-items: center; background-color: white; border-radius: 25px; width: 500px; height: 700px; box-shadow: 0 4px 10px rgba(0,0,0,0.1); }

input { padding-left: 15px; width: 460px; height: 50px; border-radius: 10px; border: 1px solid #C8D3E5; }

button { width: 460px; height: 50px; background-color: #1A5FBB; border-radius: 10px; color: white; font-weight: bold; cursor: pointer; }

hr { width: 90%; border: none; border-top: 1px solid #C8D3E5; }
```

authorization.vue

<script setup>

Продолжение приложения А

```
import logo from '../assets/images/logo.png'; import './Authorization.css'; import { ref } from 'vue'; import
{ useRouter } from 'vue-router';

const router = useRouter(); const logintxt = ref(""); const passwordtxt = ref(""); const error = ref(""); const
messageError = ref("");

const auth = async () => {

  if (!logintxt.value || !passwordtxt.value) { messageError.value = 'Заполните поля!'; return; }

  const response = await fetch(`http://localhost:5095/api/User/login?login=${logintxt.value}&password=${
passwordtxt.value}`);

  const data = await response.json();

  if (response.ok) {

    localStorage.setItem('userId', data.userId); localStorage.setItem('roleId', data.roleId);
    localStorage.setItem('fullname', data.fullname);

    const routes = { 1: '/usersList', 2: '/addorder', 3: '/mytrips', 4: '/neworder' }; router.push(routes[data.roleId]);

  } else { error.value = data.message; setTimeout(() => error.value = "", 5000); }

}

</script>

<template> <div class="auth"><div class="label"><form id="background" @submit.prevent="auth">

 <h1 id="translog">ТРАНС<span id="log">ЛОГ</span></h1><p id="uchet">УЧЕТ
ЗАЯВОК</p><hr>

<p class="textinput">Логин</p><input v-model="logintxt" type="text"><p class="textinput">Пароль</p><input
v-model="passwordtxt" type="password">

<p v-if="error" style="color:red">{{ error }}</p><button type="submit">Войти</button>

</form></div></div>

</template>

register.css

* { margin: 0; padding: 0; box-sizing: border-box; }

#app { display: flex; flex-direction: column; }

html, body { height: 100%; background-color: #FFFAFA; }

.auth { display: flex; background-color: white; justify-content: center; align-items: center; min-height: 100vh; }

#background { display: flex; flex-direction: column; align-items: center; width: 100%; gap: 10px; background-color:
white; border-radius: 15px; height: 850px; box-shadow: 0 4px 10px rgba(0,0,0,0.1); }
```

Продолжение приложения А

```
img { width: 400px; height: 250px; margin-top: 10px; justify-content: center; align-items: center; }

#translog { display: flex; align-items: center; justify-content: center; font-family: Inter; font-weight: bold; font-size: 32px; line-height: auto; letter-spacing: 2%; color: #1D2D50; margin-top: -70px; }

#log { font-family: Inter; font-weight: bold; font-size: 32px; line-height: auto; letter-spacing: 2%; color: #1A5FBB }

.label { display: flex; justify-content: center; align-items: center; background-color: white; border-radius: 25px; width: 500px; height: 700px; box-shadow: 0 4px 10px rgba(0,0,0,0.1); }

#uchet { margin-top: -10px; display: flex; justify-content: center; align-items: center; color: #7A8BA8; font-weight: bold; font-family: Inter; font-size: 14px; }

input { padding-left: 15px; width: 460px; height: 50px; background-color: #F6F9FC; border-radius: 10px; border: 1px solid #C8D3E5; }

.textinput { width: auto; align-self: flex-start; margin-left: 20px; text-align: left; color: #5A6A84; font-family: Inter; font-size: 16px; font-weight: bold; }

#noaccount { display: flex; align-items: center; justify-content: center; text-align: right; color: #1A5FBB; font-family: Inter; font-size: 17px; width: 450px; }

button { width: 460px; height: 50px; background-color: #1A5FBB; border-radius: 10px; border: none; font-family: Inter; font-size: 20px; font-weight: bold; color: white; }

#line { display: flex; width: 480px; height: 2px; background-color: #D9D9D9; }

hr { width: 90%; border: none; border-top: 1px solid #C8D3E5; }

.divhorizontal { display: flex; flex-direction: row; justify-content: center; align-items: center; width: 460px; gap: 15px; }

.inputmini { width: 223px; height: 50px; background-color: #F6F9FC; border-radius: 10px; border: 1px solid #C8D3E5; }

.div1 { display: flex; align-items: left; flex-direction: column; justify-content: left; }
```

register.vue

<script setup>

```
import router from '@/router';
import logo from '../assets/images/logo.png'
import './register.css'
import { ref, computed, watch } from 'vue'
const surnametext = ref("")
const nametext = ref("")
const otchestvo = ref("")
const logintext = ref("")
const passwordtext = ref("")
const numberphone = ref("")
```

Продолжение приложения А

```
const error = ref("")
const messageError = ref("")
const fullname = computed(() => `${surnametxt.value} ${nametxt.value} ${otchestvo.value}`)
watch(numberphone, (newValue, oldValue) => {
  if (!newValue.startsWith('+7')) {
    numberphone.value = '+7' + newValue.replace(/[^0-9]/g, "")
    let cleaned = '+7' + newValue.slice(2).replace(/[^0-9]/g, "")
    if (cleaned.length > 12) {
      cleaned = cleaned.slice(0, 12)
    }
    if (cleaned !== newValue) { numberphone.value = cleaned } }) const register = async () => {
  if (!surnametxt.value || !nametxt.value || !logintxt.value || !passwordtxt.value || !numberphone.value) {
    messageError.value = 'Заполните все поля!' return }
  const phoneRegex = /^\\+7\\d{10}$/ if (!phoneRegex.test(numberphone.value)) {
    messageError.value = 'Введите корректный номер телефона' setTimeout(() => { messageError.value = "" }, 5000)
    return } const response = await fetch(
    `http://localhost:5095/api/User/register`,
    {method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify({
      username: logintxt.value,
      password: passwordtxt.value,
      fullname: fullname.value,
      numberphone: numberphone.value,
      roleId: 4, isActive: 1}) })
  const data = await response.json() if (response.ok) { router.push('/authorization') } else { messageError.value =
    data.message } console.log(data) }
  const handlePhoneFocus = (event) => {
    if (!numberphone.value || numberphone.value === "") { numberphone.value = '+7'
    } setTimeout(() => { if (event.target) { event.target.setSelectionRange(event.target.value.length,
    event.target.value.length) } }, 0) }
  const handlePhoneInput = (event) => {
    let value = event.target.value
    if (!value.startsWith('+7')) {
      numberphone.value = '+7' return }
    let cleaned = '+7' + value.slice(2).replace(/[^0-9]/g, "")
    if (cleaned.length > 12) {
      cleaned = cleaned.slice(0, 12)
    }
    numberphone.value = cleaned
  }
</script>

<template>
<div class="auth">
<div class="label">
<form id="background" @submit.prevent="register">

<h1 id="translog"><span>ТРАНС</span><span id="log">ЛОГ</span></h1>
<p id="uchet">РЕГИСТРАЦИЯ НОВОГО ПОЛЬЗОВАТЕЛЯ</p> <hr>
<p class="textinput">ЛИЧНЫЕ ДАННЫЕ</p>
<div class="divhorizontal"> <div class="div1">
<p class="textinput">Фамилия</p>
<input class="inputmini" v-model="surnametxt" placeholder="Иванов" type="text" >
</div> <div class="div1">
```

Продолжение приложения А

```
<p class="textinput">Имя</p>
<input class="inputmini" v-model="nametxt" placeholder="Иван" type="text" > </div>
</div> <div class="divhorizontal"> <div class="div1">
<p class="textinput">Отчество</p>
<input placeholder="Отчество" v-model="otchestvo" class="inputmini" type="text"> </div>
<div class="div1"> <p class="textinput">Номер телефона</p> <input type="text" class="inputmini" v-
model="numberphone" placeholder="+7 (900) 321-67-52" maxlength="12" @focus="handlePhoneFocus"
@input="handlePhoneInput"> </div> </div>
<p class="textinput">Данные для входа</p> <p class="textinput">Логин</p>
<input placeholder="Придумайте логин" v-model="logintxt" type="text" >
<p class="textinput">Пароль</p>
<input placeholder="*****" v-model="passwordtxt" type="password">
<button type="submit">Регистрация</button>
<p v-if="messageError" style="color: red; font-size: 14px; margin-bottom: 0px;">{{ messageError }}</p>
<a id="noaccount" @click="$router.push('/authorization')" >Есть аккаунт? Войдите</a>
</form> </div> </div>
</template>
```

historyorder.vue

```
<script setup>
```

```
import router from '@router';
import logo from '../assets/images/logo.png'
import { ref, computed, onMounted } from 'vue'
import plus from '../assets/images/plus.png'
import time from '../assets/images/time.png'
import history from '../assets/images/history.png'
const fullname = localStorage.getItem('fullname')
const logout = () => {
  localStorage.clear()
  router.push('/authorization') }
const departurepoint = ref("")
const arrivalpoint = ref("")
const weight = ref("")
const volumem3 = ref("")
const description = ref("")
const orders = ref([])
const statusOrder = {
  'Ожидает оплаты': 0,
  'Ожидание': 1,
  'Принято': 2,
  'Выполняется': 3,
  'Доставлено': 4,
  'Отменено': 5}
onMounted(async () => {{
  const userID = parseInt(localStorage.getItem('userId'))
  const response = await fetch(
    `http://localhost:5095/api/Order/GetHistory?Userid=${userID}`)
  const data = await response.json()
```


Продолжение приложения А

```
orders.value = data.sort((a, b) => (statusOrder[a.status] ?? 99) - (statusOrder[b.status] ?? 99))
console.log(data)}
})const initials = computed(() => {
if (!fullname) return "
return fullname.split(' ').map(word => word[0]).join("").toUpperCase())
const totalOrders = computed( () => orders.value.length)
const totalwork = computed ( () => orders.value.filter(o => o.status == 'Выполняется').length)
const totalend = computed ( () => orders.value.filter(o => o.status == 'Доставлено').length)
const totalwait = computed ( () => orders.value.filter(o => o.status == 'Ожидание').length)
</script>
<template>
<div class="layout">
<div class="sidebar">
<div class="logo">
<h1>ТРАНС<span>ЛОГ</span></h1>
<p>ГРУЗОПЕРЕВОЗКИ</p>
</div><div class="user"><div class="avatar">{{ initials }}</div><div><p class="user-name">{{ fullname }}</p>
<p class="user-role">Клиент</p></div></div>

<div class="menu"><div class="podmenu" @click="$router.push('/neworder')"> <a
class="menu-item active" >Новая заявка</a>
</div>
<div class="podmenu" @click="$router.push('/statusorder')">

<a class="menu-item">Статус заявок</a>
</div >
<div class="podmenu_active">

<a class="menu-item">История заявок</a></div></div><hr>
<a class="logout" @click="logout">Выйти из системы</a></div>
<div class="content"><div class="topbar">История заявок</div><div class="cardhistory" style="margin:
30px;"><div class="row"><div class="lab"><p class="lab_title">Всего заявок</p><p style="color: black; font-
size: 34px; margin-bottom: -50px; margin-left: 25px; ">{{ totalOrders }}</p><p class="lab_other">За всё
время</p></div><div class="lab"><p class="lab_title">В работе</p><p style="color: black; font-size: 34px;
margin-bottom: -50px; margin-left: 25px; ">{{ totalwork }}</p><p class="lab_other">активных</p></div>
<div class="lab">
<p class="lab_title">Доставлено</p>
<p style="color: black; font-size: 34px; margin-bottom: -50px; margin-left: 25px; ">{{ totalend }}</p>
<p class="lab_other">завершено</p></div>
<div class="lab">
<p class="lab_title">Ожидает</p>
<p style="color: black; font-size: 34px; margin-bottom: -50px; margin-left: 25px; ">{{ totalwait }}</p>
<p class="lab_other">в обработке</p></div></div>
<h2 style="margin-top: 30px;">История всех заявок</h2>
<div style="overflow-x: auto; overflow-y: auto !important; max-height: 460px !important; ">
<table><thead><tr>
<td>№ ЗАЯВКИ</td><td>ДАТА</td><td>МАРШРУТ</td><td>Отправление → Прибытие</td><td>ГРУЗ
(Т)</td><td>СТАТУС</td></tr></thead><tbody><tr v-for="order in orders" :key="order.orderId">
<td>#{{ order.orderId }}</td>
<td>{{ new Date(order.receivedAt).toLocaleDateString('ru-RU') }}</td>
```

Продолжение приложения А

```
<td>{{ order.departurePoint }} → {{ order.arrivalPoint }}</td>
<td>{{ order.departureTime ? new Date(order.departureTime).toLocaleDateString('ru-RU') : 'Ожидайте' }} →
{{ order.arrivalTime ? new Date(order.arrivalTime).toLocaleDateString('ru-RU') : 'Ожидайте' }} </td>
<td>{{ order.weight }}</td><td><span
class="status":class="{
waiting: order.status === 'Ожидание',
accepted: order.status === 'Принято',
paying: order.status === 'Ожидает оплаты',
progress: order.status === 'Выполняется',
done: order.status === 'Доставлено',
cancel: order.status === 'Отменено'}">{{ order.status }}</span></td></tr></tbody></table></div>
</div></div></div>
</template>
```

neworder.vue

```
<script setup>
```

```
import router from '@router';
import logo from '../assets/images/logo.png'
import { ref, computed } from 'vue'
import plus from '../assets/images/plus.png'
import time from '../assets/images/time.png'
import history from '../assets/images/history.png'
```

```
const messageAlert = ref("")
const departurepoint = ref("")
const arrivalpoint = ref("")
const weight = ref("")
const volumem3 = ref("")
const description = ref("")
const colormessage = ref("")
const initials = computed(() => {
if (!fullname) return ""
return fullname.split(' ').map(word => word[0]).join("").toUpperCase()})
```

```
const order = async () =>{
if (!departurepoint.value || !arrivalpoint.value || !weight.value || !volumem3.value){
colormessage.value = 'red'
messageAlert.value = 'Заполните все поля'
setTimeout(() => {messageAlert.value = ""}, 5000)return}
```

```
const response = await fetch(
'http://localhost:5095/api/Order/CreateOrder',{method: 'POST',
headers: {'Content-Type': 'application/json'},body: JSON.stringify({
userId: parseInt(localStorage.getItem('userId')),
DeparturePoint: departurepoint.value,
ArrivalPoint: arrivalpoint.value,
Weight: parseFloat(weight.value),
Volumem3: parseFloat(volumem3.value),
```

Продолжение приложения А

```
Description: description.value,
Status: 'Ожидание',
ReceivedAt: new Date().toISOString()}}))
const data = await response.json()
console.log(data)if (response.ok)
{departurepoint.value = "
arrivalpoint.value = "
weight.value = "
volumem3.value = "
description.value = "
colormessage.value = 'green'
messageAlert.value = data.message
setTimeout(() => {messageAlert.value = ""}, 10000)}
const fullname = localStorage.getItem('fullname')
const logout = () => {
localStorage.clear()
router.push('/authorization')}
</script>
<template>
<div class="layout">
<div class="sidebar">
<div class="logo">
<h1>ТРАНС<span>ЛОГ</span></h1>
<p>ГРУЗОПЕРЕВОЗКИ</p>
</div>
<div class="user"><div class="avatar">{{ initials }}</div>
<div><p class="user-name">{{ fullname || 'Нет' }}</p><p class="user-role">Клиент</p></div></div>
<div class="menu"><div class="podmenu_active">
<a class="menu-item active" >Новая заявка</a></div>
<div class="podmenu" @click="$router.push('/statusorder')">

<a class="menu-item" >Статус заявок</a></div >
<div class="podmenu" @click="$router.push('/historyorder')">

<a class="menu-item">История заявок</a></div></div><hr>
<a class="logout" @click="logout">Выйти из системы</a></div>
<form @submit.prevent="order">
<div class="content">
<div class="topbar">Новая заявка</div>
<div class="card">
<p class="card-title">Оформление заявки на грузоперевозку</p>
<h2>МАРШРУТ</h2>
<div class="row">
<div class="field">
<label style="font-size: 18px;">Пункт отправления</label>
<input type="text" v-model="departurepoint" placeholder="Москва"></div>
<div class="field">
<label style="font-size: 18px;" >Пункт прибытия</label>
<input type="text" v-model="arrivalpoint" placeholder="Уфа"></div></div>
```

Продолжение приложения А

```
<h2>ПАРАМЕТРЫ ГРУЗА</h2>
<div class="row">
  <div class="field">
    <label style="font-size: 18px;">Вес груза (тонн)</label>
    <input type="number" v-model="weight" step="0.01" placeholder="0.5">
  </div>
  <div class="field">
    <label style="font-size: 18px;">Объем груза (м2)</label>
    <input type="number" v-model="volumem3" step="0.01" placeholder="1.0">
  </div></div>
  <div class="field"><label style="font-size: 18px;">Описание груза (необязательно)</label>
  <textarea rows="5" v-model="description"></textarea></div>
  <p v-if="messageAlert" :style="{color: colormessage, fontSize: '20px'}"> {{ messageAlert }}</p>
  <button class="submit" style="font-size: 18px;">Подать заявку</button></div></div></form></div>
</template>
```

statusorder.vue

```
<script setup>

import router from '@router';
import logo from '../assets/images/logo.png'
import { ref, computed, onMounted } from 'vue'
import plus from '../assets/images/plus.png'
import time from '../assets/images/time.png'
import history from '../assets/images/history.png'
const departurepoint = ref("")
const arrivalpoint = ref("")
const weight = ref("")
const volumem3 = ref("")
const description = ref("")
const orders = ref([])
const statusOrder = { 'Ожидает оплаты': 0, 'Ожидает': 1, 'Принято': 2, 'Выполняется': 3, }
const sortOrders = (data) => {
  return data.sort((a, b) => statusOrder[a.status] - statusOrder[b.status])}
onMounted(async () => {
  const userID = parseInt(localStorage.getItem('userId'))
  const response = await fetch(
    `http://localhost:5095/api/Order/GetHistory?Userid=${userID}`)
  const data = await response.json()
  orders.value = data.filter(o => o.status !== 'Доставлено' && o.status !== 'Отменено')
  console.log(data)
  orders.value = sortOrders(data.filter(o => o.status !== 'Доставлено' && o.status !== 'Отменено'))})
const logout = () => {
  localStorage.clear()
  router.push('/authorization')}
const fullname = localStorage.getItem('fullname')
const orderID = ref("")
const cancel = async (orderId) =>{
  const response = await fetch(`http://localhost:5095/api/Order/UpdateOrder?OrderId=${orderId}`, {
```

Продолжение приложения А

```
method: 'PUT', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify( { Status: 'Отменено' } ))
const userID = parseInt(localStorage.getItem('userId'))
const res = await fetch(`http://localhost:5095/api/Order/GetHistory?Userid=${userID}`)
const data = await res.json()
orders.value = sortOrders(data.filter(o => o.status !== 'Доставлено' && o.status !== 'Отменено'))
const initials = computed(() => { if (!fullname) return 'return fullname.split(' ').map(word =>
word[0]).join('').toUpperCase() })
const showPaymentModal = ref(false)
const selectedOrder = ref(null)
function payOrder(orderId) {
selectedOrder.value = orders.value.find(o => o.orderId === orderId);
showPaymentModal.value = true; }
async function confirmPay() { if (!selectedOrder.value) return;
await fetch(`http://localhost:5095/api/Order/UpdateOrder?OrderId=${selectedOrder.value.orderId}`, {
method: 'PUT', headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ Status: 'Принято' }) });
const userID = parseInt(localStorage.getItem('userId'));
const res = await fetch(`http://localhost:5095/api/Order/GetHistory?Userid=${userID}`);
const data = await res.json();
orders.value = sortOrders(data.filter(o => o.status !== 'Доставлено' && o.status !== 'Отменено'));
showPaymentModal.value = false;
selectedOrder.value = null; }
function cancelPayModal() {
showPaymentModal.value = false
selectedOrder.value = null }
async function downloadReceipt(orderId) {
try { const response = await fetch(`http://localhost:5095/api/Order/GetReceipt?orderId=${orderId}`);
if (!response.ok) {
alert('Ошибка при скачивании чека');
return;
}
const blob = await response.blob();
const url = window.URL.createObjectURL(blob);
const link = document.createElement('a');
link.href = url;
link.setAttribute('download', `check_${orderId}.pdf`);
document.body.appendChild(link);
link.click();
link.remove();
window.URL.revokeObjectURL(url); } catch (e) { alert('Ошибка при скачивании чека'); } }
</script>
<template><div class="layout">
<div class="sidebar">
<div class="logo">
<h1>ТРАНС<span>ЛОГ</span></h1>
<p>ГРУЗОПЕРЕВОЗКИ</p></div>
<div class="user">
<div class="avatar">{{ initials }}</div><div>
<p class="user-name">{{ fullname }}</p>
<p class="user-role">Клиент</p></div> </div> <div class="menu">
```

Продолжение приложения А

```
<div class="podmenu" @click="$router.push('/neworder')"> 
<a class="menu-item" > Новая заявка</a></div> <div class="podmenu_active" >  <a
class="menu-item" >Статус заявок</a> </div>
<div class="podmenu" @click="$router.push('/historyorder')">
 <a class="menu-item">История заявок</a> </div></div> <hr> <a class="logout"
@click="logout">Выйти из системы</a></div>
<div class="content">
<div class="topbar">Статус заявки</div> <div class="card">
<h2 id="titleorder" style="margin-top: -30px;" >Текущие заявки</h2>
<div style="overflow-y: auto; max-height: 600px;"><table><thead><tr>
<td>№ ЗАЯВКИ</td><td>ДАТА</td><td>МАРШРУТ</td><td>Отправление → Прибытие</td><td>ГРУЗ
(T)</td><td>СТАТУС</td></tr></thead><tbody>
<tr v-for="order in orders" :key="order.orderId">
<td>#{{ order.orderId }}</td>
<td>{{ new Date(order.receivedAt).toLocaleDateString('ru-RU') }}</td>
<td>{{ order.departurePoint }} → {{ order.arrivalPoint }}</td>
<td>{{ order.departureTime ? new Date(order.departureTime).toLocaleDateString('ru-RU') : 'Ожидайте' }} →
{{ order.arrivalTime ? new Date(order.arrivalTime).toLocaleDateString('ru-RU') : 'Ожидайте' }}
</td><td>{{ order.weight }}</td> <td> <span
class="status" :class="{ waiting: order.status === 'Ожидание', accepted: order.status === 'Принято',
paying: order.status === 'Ожидает оплаты', progress: order.status === 'Выполняется',
done: order.status === 'Доставлено', cancel: order.status === 'Отменено' }" > {{ order.status }} </span></td><td>
<button v-if="order.status === 'Принято'" @click="downloadReceipt(order.orderId)" class="btn-receipt"
style="width: 150px; font-size: 14px; height: 40px; background: #2196f3; color: white; border: none; padding: 8px
16px; border-radius: 25px; cursor: pointer; margin-top: 6px; margin-right: 5px"> Квитанция </button> <button v-
if="order.status === 'Ожидает оплаты'" @click="payOrder(order.orderId)"
class="btn-pay" style="width: 150px; font-size: 14px; height: 40px; background: #2ecc71; color: white; border:
none; padding: 8px 16px; border-radius: 25px; cursor: pointer;"> Оплатить </button> <button
v-if="order.status === 'Ожидание' || order.status === 'Принято' || order.status === 'Ожидает оплаты'"
@click="cancel(order.orderId)" class="btn-cancel" style="width: 150px; font-size: 14px; height: 40px; background:
#e74c3c; color: white; border: none; padding: 8px 16px; border-radius: 25px; cursor: pointer; margin-left: 5px;">
Отменить </button></td></tr></tbody></table></div> </div> </div> </div>
<div v-if="showPaymentModal && selectedOrder" class="showStatusModal"> <div class="modal">
<h2 style="font-size:22px;margin-bottom:16px;">Оплата заказа</h2>
<div class="simple-field"> <label> Откуда:
<input type="text" readonly :value="selectedOrder.departurePoint" />
</label> <label> Куда:
<input type="text" readonly :value="selectedOrder.arrivalPoint" /> </label> <label> Объем (м³):
<input type="text" readonly :value="selectedOrder.volumeM3 || selectedOrder.volumem3 || "" />
</label> <label> Вес (кг):
<input type="text" readonly :value="selectedOrder.weight || selectedOrder.volumem3 || "" />
</label> <label> Дистанция (км):
<input type="text" readonly :value="selectedOrder.distanceKm || "" />
</label> <label> Цена (₽):
<input type="text" readonly :value="selectedOrder.price || "" /></label> </div>
<div style="margin-top:28px;display:flex;gap:16px;justify-content:flex-end;">
<button @click="confirmPay" style="background:#27ae60;color:white;padding:9px 28px;border:none;border-
radius:7px;font-size:17px;cursor:pointer;">Оплатить</button>
<button @click="cancelPayModal" style="background:#eee;color:#222;padding:9px 20px;border:none;border-
radius:7px;font-size:16px;cursor:pointer;">Отменить</button> </div> </div></div>
```

Продолжение приложения А

</template>

mytransport.vue

<script setup>

```
import router from '@router'
import { ref, onMounted, computed } from 'vue'
import order from '../assets/images/order.png'
import transport from '../assets/images/transport.png'
const fullname = localStorage.getItem('fullname')
const myVehicle = ref(null)

onMounted(async () => {
  const userId = localStorage.getItem('userId')
  const response = await fetch('http://localhost:5095/api/Driver/GetDrivers')
  const drivers = await response.json()
  const myDriver = drivers.find(d => d.userId == userId)
  if (myDriver) myVehicle.value = myDriver.vehicle })
const logout = () => { localStorage.clear() router.push('/authorization')}
const initials = computed(() => { if (!fullname) return ''
return fullname.split(' ').map(word => word[0]).join('').toUpperCase() })
```

</script>

<template>

```
<div class="layout"> <div class="sidebar"> <div class="logo">
<h1>ТРАНСПОРТ</h1> <p>ГРУЗОПЕРЕВОЗКИ</p> </div> <div class="user">
<div class="avatar">{{ initials }}</div> <div> <p class="user-name">{{ fullname }}</p> <p class="user-
role">Водитель</p>
</div> </div> <div class="menu"> <div class="podmenu" @click="$router.push('/mytrips')">
 <a class="menu-item">Мои рейсы</a> </div> <div class="podmenu_active">
 <a class="menu-item">Транспорт</a> </div> </div>
<hr> <a class="logout" @click="logout">Выйти из системы</a> </div> <div class="content">
<div class="topbar">Мой транспорт</div> <div class="card"> <h2>Мой автомобиль</h2> <div v-
if="myVehicle" style="margin-top: 20px;"> <table> <thead> <tr> <td>Гос. номер</td>
<td>Марка</td> <td>Грузоподъёмность (кг)</td> <td>Объём (м³)</td> <td>Тип</td></tr></thead> <tbody><tr>
<td>{{ myVehicle.licensePlate }}</td>
<td>{{ myVehicle.brand }} {{ myVehicle.model }}</td>
<td>{{ myVehicle.payloadKg }}</td>
<td>{{ myVehicle.volumeM3 }}</td>
<td>{{ myVehicle.vehicleType?.name ?? 'Не указан' }}</td>
</tr> </tbody></table> </div><div v-else style="margin-top: 20px; color: #7a8ba8; font-size: 18px;">
Транспорт не назначен</div></div></div></div>
</template>
```

mytrips.vue

<script setup>

```
import router from '@router';
import logo from '../assets/images/logo.png'
import { ref, computed, onMounted } from 'vue'
```

Продолжение приложения А

```
import order from '../assets/images/order.png'
import people from '../assets/images/people.png'
import transport from '../assets/images/transport.png'
const logout = () => {localStorage.clear()}
router.push('/authorization')}
const orders = ref([])
onMounted(async () => { const response = await fetch(
`http://localhost:5095/api/Order/GetOrder`)
const data = await response.json()
const userId = localStorage.getItem('userId')
orders.value = data.filter(o =>
o.vehicle?.drivers?.some(d => d.userId == userId) )
console.log(data) })
const fullname = localStorage.getItem('fullname')
const showStatusModal = ref(false)
const newStatus = ref("")
const currentOrder = ref(null)
const openStatusModal = (order) => { currentOrder.value = order
newStatus.value = order.status
showStatusModal.value = true } const saveStatus = async () => {
const response = await fetch(
`http://localhost:5095/api/Order/UpdateOrder?OrderId=${currentOrder.value.orderId}`,
{method: 'PUT',headers: {'Content-Type': 'application/json'}, body: JSON.stringify({status: newStatus.value}) })
showStatusModal.value = false
const res = await fetch('http://localhost:5095/api/Order/GetOrder')
const data = await res.json()
const userId = localStorage.getItem('userId')
orders.value = data.filter(o =>
o.vehicle?.drivers?.some(d => d.userId == userId) ))
const showDescModal = ref(false)
const currentDesc = ref("")
const openDescModal = (order) =>{currentDesc.value = order.description || 'Описание не указано'
showDescModal.value = true}
const initials = computed(() => {
if (!fullname) return ""
return fullname
.split(' ').map(word => word[0]).join("").toUpperCase() })
const showCheckModal = ref(false)
const currentCheck = ref(null)
const openCheck = (order) => { currentCheck.value = order
showCheckModal.value = true }
const printCheck = () => { window.print()}
async function downloadReceipt(orderId) {
try { const response = await fetch(`http://localhost:5095/api/Order/GetNakladnya?orderId=${orderId}`);
if (!response.ok) { alert('Ошибка при скачивании чека'); return; }
const blob = await response.blob();
const url = window.URL.createObjectURL(blob);
const link = document.createElement('a');
link.href = url;
link.setAttribute('download', `nakladnya_${orderId}.pdf`);
```


Продолжение приложения А

```
document.body.appendChild(link);
link.click();
link.remove(); window.URL.revokeObjectURL(url);
} catch (e) { alert('Ошибка при скачивании чека'); }}
</script>
<template>
<div class="layout"> <div class="sidebar"> <div class="logo">
<h1>ТРАНС<span>ЛОГ</span></h1> <p>ГРУЗОПЕРЕВОЗКИ</p> </div>
<div class="user">
<div class="avatar">{{ initials }}</div> <div>
<p class="user-name"> {{ fullname }}</p>
<p class="user-role">Водитель</p>
</div>
</div>
<div class="menu">
<div class="podmenu_active">

<a class="menu-item active">Мои рейсы</a>
</div>
<div class="podmenu" @click="$router.push('/mytransport')">

<a class="menu-item">Транспорт</a></div></div><hr><a class="logout" @click="logout">Выйти из
системы</a> </div>
<div class="content"><div class="topbar">Мои рейсы</div><div class="card"><h2>Мои рейсы</h2><div
style="overflow-x: auto; max-width: 100%;">
<div style="overflow-y: auto; max-height: 640px; text-wrap:
nowrap;"><table><thead><tr><td>Маршрут</td><td>Отправление → Прибытие</td> <td>Расстояние</td>
<td>Груз (Т)</td> <td>Контакты</td> <td>Статус</td> </tr> </thead>
<tbody>
<tr v-for="o in orders" :key="o.orderId"> <td style="text-wrap: nowrap;">{{ o.departurePoint ?? '-' }} →
{{ o.arrivalPoint }}</td>
<td style="text-wrap: nowrap;">{{ o.departureTime ? new Date(o.departureTime).toLocaleDateString('ru-RU') :
'Ожидайте'}} → {{ o.arrivalTime ? new Date(o.arrivalTime).toLocaleDateString('ru-RU') : 'Ожидайте'}}</td>
<td>{{ o.distanceKm || '-' }}</td> <td>{{ o.weight }}</td> <td>{{ o.user?.numberphone ?? '-' }}</td> <td> <span
class="status" :class="{ waiting: o.status === 'Ожидает', progress: o.status === 'Выполняется',
done: o.status === 'Доставлено', cancel: o.status === 'Отменено' }" > {{ o.status }} </span></td> <td>
<div style="display: flex; flex-direction: column; gap: 5px;">
<button class="manbtn" @click="openStatusModal(o)">Статус</button>
<button class="manbtn" @click="openDescModal(o)">Описание</button>
<button class="manbtn" @click="downloadReceipt(o.orderId)" :disabled="o.status !== 'Доставлено'"
:style="o.status !== 'Доставлено' ? 'opacity: 0.4; cursor: not-allowed;'" : "">
Накладная
</button></div></td></tr> </tbody> </table>
</div></div></div> </div></div>
<div v-if="showStatusModal" class="showStatusModal"><div>
<p>Изменить статус #1</p> <p>Новый статус</p>
<select v-model="newStatus" class="simple-select">
<option value="Выполняется">Выполняется</option>
<option value="Доставлено">Доставлено</option> </select> <div class="simple-buttons">
<button class="simple-cancel" @click="showStatusModal = false">Отмена</button>
```

Продолжение приложения А

```
<button class="simple-save" @click="saveStatus">Сохранить</button>
</div> </div></div>
<div v-if="showDescModal" class="showStatusModal"><div>
<p>Описание заказа</p>
<p style="font-size: 16px; color: black;">{{ currentDesc }}</p>
<div class="simple-buttons">
<button class="simple-save" @click="showDescModal = false">Закрыть</button>
</div></div></div>
</template>
```

addorder.vue

```
<script setup>
```

```
import router from '@router';
import logo from '../assets/images/logo.png'
import { ref, computed, onMounted, watch } from 'vue'
import order from '../assets/images/order.png'
import people from '../assets/images/people.png'
import transport from '../assets/images/transport.png'

const showTripModal = ref(false)
const currentOrder = ref(null)
const tripData = ref({
  vehicleId: null,
  from: "",
  to: "",
  distanceKm: 0, price: 0, departureDate: "", arrivalDate: ""})
const vehicleRates = { 1: 50, 2: 80, }
const calculatePrice = () => {
  const distance = parseFloat(tripData.value.distanceKm)
  const vehicleId = tripData.value.vehicleId
  if (distance > 0 && vehicleId) {
    const selectedVehicle = vehicles.value.find(v => v.vehicleId === vehicleId)
    if (selectedVehicle) {
      const rate = selectedVehicle.vehicleType?.pricePerKm || vehicleRates[selectedVehicle.vehicleTypeId] || 50
      tripData.value.price = distance * rate
      console.log(`Расчет: ${distance} км * тариф ${rate} руб = ${tripData.value.price}`)} else {tripData.value.price = 0}}
  watch(() => tripData.value.distanceKm, calculatePrice)
  watch(() => tripData.value.vehicleId, calculatePrice)
  const errorMessage = ref("") const saveTrip = async () =>{
    if (!tripData.value.vehicleId || !tripData.value.departureDate || !tripData.value.arrivalDate || !tripData.value.distanceKm) {
      errorMessage.value = 'Заполните все поля!' setTimeout(() => {errorMessage.value = ""}, 5000)
      return}
    const response = await fetch(
      `http://localhost:5095/api/Order/UpdateOrder?OrderId=${currentOrder.value.orderId}`, {
      method: "PUT", headers: { 'Content-Type': 'application/json'}, body: JSON.stringify({
        Status: 'Ожидает оплаты',
```

Продолжение приложения А

```
DepartureTime: tripData.value.departureDate,
ArrivalTime: tripData.value.arrivalDate,
vehicleId: tripData.value.vehicleId,
Distance_km: tripData.value.distanceKm,
Price: tripData.value.price }}})
const data = await response.json()
console.log(data)
showTripModal.value = false
const res = await fetch('http://localhost:5095/api/Order/GetOrder')
orders.value = sortOrders(await res.json())
const openTripModal = (order) => {
  currentOrder.value = order
  tripData.value = {
    vehicleId: order.vehicleId || null,
    from: order.departurePoint,
    to: order.arrivalPoint,
    distanceKm: order.distanceKm || 0,
    departureDate: order.departureTime || '',
    arrivalDate: order.arrivalTime || '',
    Price: order.price || 0, }
  console.log('Полный объект заказа:', order)
  console.log('distance_km:', order.distanceKm)
  console.log('DistanceKm:', order.distanceKm)
  showTripModal.value = true
  calculatePrice() }
const selectedDriverName = computed(() => {
  if (!tripData.value.vehicleId) return ''
  const vehicle = vehicles.value.find(v => v.vehicleId === tripData.value.vehicleId)
  if (!vehicle || !vehicle.drivers?.[0]?.user) return 'Водитель не привязан'
  return vehicle.drivers[0].user.fullName })
const departurepoint = ref('')
const arrivalpoint = ref('')
const weight = ref('')
const volumem3 = ref('')
const description = ref('')
const orders = ref([])
const drivers = ref([])
const vehicles = ref([])
onMounted(async () => { {
  const userID = parseInt(localStorage.getItem('userId'))
  const response = await fetch(
    `http://localhost:5095/api/Order/GetOrder`)
  const data = await response.json()
  orders.value = sortOrders(data)
  console.log(data)
  const response2 = await fetch('http://localhost:5095/api/Driver/GetDrivers')
  drivers.value = await response2.json()
  console.log('drivers:', drivers.value)
  const response3 = await fetch('http://localhost:5095/api/Vehicle/GetTransport')
  vehicles.value = await response3.json()
}
```

Продолжение приложения А

```
console.log('vehicles:', vehicles.value) })
const logout = () => {
  localStorage.clear()
  router.push('/authorization') }
const newStatus = ref("")
const showStatusModal = ref(false)
const openStatusModal = (order) => {
  currentOrder.value = order
  newStatus.value = order.status
  showStatusModal.value = true }

const saveStatus = async () => {
  const response = await fetch(
    `http://localhost:5095/api/Order/UpdateOrder?OrderId=${currentOrder.value.orderId}`, {
    method: 'PUT', headers: {'Content-Type': 'application/json'}, body: JSON.stringify({Status: newStatus.value}) } )
  showStatusModal.value = false
  const res = await fetch('http://localhost:5095/api/Order/GetOrder')
  orders.value = sortOrders(await res.json()) }
const fullname = localStorage.getItem('fullname')
const totalOrders = computed(() => orders.value.length)
const totalwork = computed(() => orders.value.filter(o => o.status === 'Выполняется').length)
const totalend = computed(() => orders.value.filter(o => o.status === 'Доставлено').length)
const totalwait = computed(() => orders.value.filter(o => o.status === 'Ожидание').length)
const totalpaying = computed(() => orders.value.filter(o => o.status === 'Ожидает оплаты').length)
const initials = computed(() => {
  if (!fullname) return ""

  return fullname.split(' ').map(word => word[0]).join('').toUpperCase()})

const statusOrder = { 'Ожидание': 0, 'Принято': 1,
  'Ожидает оплаты': 2, 'Выполняется': 3,
  'Доставлено': 4, 'Отменено': 5 }
const sortOrders = (data) => {
  return data.sort((a, b) => statusOrder[a.status] - statusOrder[b.status])}

const today = new Date().toISOString().split('T')[0]
const availableVehicles = computed(() =>
  vehicles.value.filter(v => {
    if (!v.drivers || v.drivers.length === 0) return false
    if (currentOrder.value?.weight && v.payloadKg < currentOrder.value.weight) return false
    if (currentOrder.value?.volumeM3 && v.volumeM3 < currentOrder.value.volumeM3) return false
    return true}))
</script>
<template>
<div class="layout">
<div class="sidebar">
<div class="logo">
<h1>ТРАНС<span>ЛОГ</span></h1>
<p>ГРУЗОПЕРЕВОЗКИ</p> </div>
<div class="user"> <div class="avatar">{{ initials }}</div> <div>
```

Продолжение приложения А

```
<p class="user-name">{{ fullname }}</p>
<p class="user-role">Менеджер</p> </div> </div>
<div class="menu"> <div class="podmenu_active"> 
<a class="menu-item"> Заявки</a> </div>
<div class="podmenu" @click="$router.push('/trips')"> 
<a class="menu-item"> Рейсы</a> </div>
<div class="podmenu" @click="$router.push('/drivers')">

<a class="menu-item"> Водители </a>
</div></div><hr>
<a class="logout" @click="logout">Выйти из системы</a> </div>
<div class="content"> <div class="topbar">Статус заявки</div>
<div class="card"> <div class="row"> <div class="lab">
<p class="lab_title">Всего заявок</p>
<p style="color: black; font-size: 34px; margin-bottom: -50px; margin-left: 25px; ">{{ totalOrders }}</p>
<p class="lab_other">За всё время</p> </div> <div class="lab"> <p class="lab_title">Новых</p>
<p style="color: black; font-size: 34px; margin-bottom: -50px; margin-left: 25px; ">{{ totalwait }}</p>
<p class="lab_other">требуют обработки</p> </div> <div class="lab">
<p class="lab_title">В работе</p>
<p style="color: black; font-size: 34px; margin-bottom: -50px; margin-left: 25px; ">{{ totalwork }}</p>
<p class="lab_other">активных</p> </div> <div class="lab">
<p class="lab_title">Закрыто</p>
<p style="color: black; font-size: 34px; margin-bottom: -50px; margin-left: 25px; ">{{ totalend }}</p>
<p class="lab_other">За всё время</p> </div> </div> <h3 id="titleorder">Все заявки</h3>
<div style="overflow-y: auto; max-height: 480px;"> <table> <thead> <tr> <td>№ ЗАЯВКИ</td>
<td>ДАТА</td> <td>МАРШРУТ</td> <td>ГПУЗ (Т)</td> <td>СТАТУС</td> </tr> </thead> <tbody>
<tr v-for="order in orders" :key="order.orderId"> <td>#{{ order.orderId }}</td>
<td>{{ new Date(order.receivedAt).toLocaleDateString('ru-RU') }}</td> <td>{{ order.departurePoint }} →
{{ order.arrivalPoint }}</td> <td>{{ order.weight }}</td> <td>
<span class="status" :class="{ waiting: order.status === 'Ожидание', paying: order.status === 'Ожидает оплаты',
accepted: order.status === 'Принято', progress: order.status === 'Выполняется', done: order.status ===
'Доставлено', cancel: order.status === 'Отменено' }"> {{ order.status }} </span> </td> <td>
<div style="display: flex; flex-direction: row; gap: 5px;">
<button class="manbtn" @click="openStatusModal(order)">Статус</button>
<button class="manbtn" @click="openTripModal(order)">Назначить</button> </div> </td> </tr>
</tbody> </table> </div> </div> </div> </div>
<div v-if="showTripModal" class="simple-modal">
<div class="simple-modal-content"> <h2>Новый рейс</h2>
<div class="simple-badge">Заявка #{{ currentOrder?.orderId }}</div> <div class="simple-row"> <div
class="simple-field"> <label>Выберите транспорт</label>
<select v-model="tripData.vehicleId" class="simple-select">
<option value="Выберите транспорт"></option>
<option v-for="v in availableVehicles" :key="v.vehicleId" :value="v.vehicleId"> {{ v.brand }} {{ v.model }}
{{ v.licensePlate }} </option> </select> </div> <div class="simple-field"> <label>Водитель</label>
<input :value="selectedDriverName" disabled placeholder="Выберите транспорт" > </div></div>
<div class="simple-row"> <div class="simple-field"> <label>Откуда</label>
<input v-model="tripData.from"> </div> <div class="simple-field"> <label>Куда</label> <input v-
model="tripData.to"> </div> </div><div class="simple-row">
<div class="simple-field"> <label>Дата отправления</label>
<input type="date" v-model="tripData.departureDate" :min="today"> </div> <div class="simple-field">
```

Продолжение приложения А

```

<label>Дата прибытия</label> <input type="date" v-model="tripData.arrivalDate" :min="today">
</div> </div> <div class="simple-field">
<label>Длина маршрута (км)</label>
<input type="number" v-model.number="tripData.distanceKm">
<label>Итоговая стоимость</label>
<input type="number" v-model="tripData.price" readonly> <p style="color: red; font-size: 14px;" v-
if="errorMessage" > {{ errorMessage }}</p> </div>
<div class="simple-buttons">
<button class="simple-cancel" @click="showTripModal = false">Отмена</button>
<button class="simple-save" @click="saveTrip">Сохранить</button>
</div> </div> </div> <div v-if="showStatusModal" class="showStatusModal" > <div>
<p>Изменить статус #1</p>
<p>Новый статус</p>
<select v-model="newStatus" class="simple-select">
<option value="Ожидание">Ожидание</option> <option value="Ожидает оплаты">Ожидает оплаты</option>
<option value="Принято">Принято</option> <option value="Выполняется">Выполняется</option>
<option value="Доставлено">Доставлено</option> <option value="Отменено">Отменено</option> </select>
<div class="simple-buttons">
<button class="simple-cancel" @click="showStatusModal = false">Отмена</button>
<button class="simple-save" @click="saveStatus">Сохранить</button> </div> </div> </div>
</template>
drivers.vue

<script setup>

import router from '@router';

import logo from '../assets/images/logo.png'

import { ref, computed, onMounted } from 'vue'
import order from '../assets/images/order.png'
import people from '../assets/images/people.png'
import transport from '../assets/images/transport.png'

const drivers = ref([]) const logout = () => { localStorage.clear() router.push('/authorization') }
onMounted(async () => {
const response = await fetch( `http://localhost:5095/api/Driver/GetDrivers` ) const data = await response.json()
drivers.value = data console.log(data) })
const fullname = localStorage.getItem('fullname')
const initials = computed(() => {
if (!fullname) return ''
return fullname.split(' ').map(word => word[0]).join('').toUpperCase()})
</script>

<template> <div class="layout"> <div class="sidebar"> <div class="logo">
<h1>ТРАНС<span>ЛОГ</span></h1><p>ГРУЗОПЕРЕВОЗКИ</p> </div>
<div class="user"> <div class="avatar">{{ initials }}</div> <div> <p class="user-name"> {{ fullname }}</p>
<p class="user-role">Менеджер</p> </div> </div><div class="menu"> <div class="podmenu">
 <a class="menu-item active" @click="$router.push('/addorder')">Заявки</a> </div>
<div class="podmenu">  <a class="menu-item" @click="$router.push('/trips')">Рейсы</a>
</div> <div class="podmenu_active">  <a class="menu-item">Водители</a>
</div> </div> <hr> <a class="logout" @click="logout">Выйти из системы</a> </div>

```

Продолжение приложения А

```
<div class="content"> <div class="topbar">Водители</div> <div class="card">
<h2>Водители и транспорт</h2> <div style="overflow-y: auto; max-height: 700px;"> <table>
<thead> <tr> <td>ФИО</td> <td>Класс</td> <td>Автомобиль</td> <td>Гос. Номер</td> <td>Груз. Т</td>
</tr> </thead> <tbody> <tr v-for="d in drivers" :key="d.driverId"> <td>{{ d.user?.fullName || '-' }}</td>
<td>{{ d.vehicle?.vehicleType?.name || '-' }}</td> <td>{{ d.vehicle?.brand || '-' }} {{ d.vehicle?.model || '-' }}</td>
<td>{{ d.vehicle?.licensePlate || '-' }}</td> <td>{{ d.vehicle?.payloadKg ? d.vehicle.payloadKg/1000 : '-' }}</td>
</tr> </tbody> </table> </div> </div> </div> </div> </template>
```

trips.vue

```
<script setup>
```

```
import router from '@router';
import logo from '../assets/images/logo.png'
import { ref, computed, onMounted } from 'vue'
import order from '../assets/images/order.png'
import people from '../assets/images/people.png'
import transport from '../assets/images/transport.png'

const orders = ref([])
const logout = () => { localStorage.clear() router.push('/authorization')}
const drivers = ref([])
const getDriverName = (vehicleId) => { if (!vehicleId) return 'Не назначен'
const driver = drivers.value.find(d => d.vehicleId === vehicleId)
return driver?.user?.fullName ?? 'Не назначен' }
```

```
onMounted(async () => { const response = await fetch(
`http://localhost:5095/api/Order/GetOrder`)
const data = await response.json() orders.value = data
console.log(data)
const response2 = await fetch('http://localhost:5095/api/Driver/GetDrivers')
drivers.value = await response2.json() })
const fullname = localStorage.getItem('fullname') const initials = computed(() => {
if (!fullname) return ''
return fullname.split(' ').map(word => word[0]).join('').toUpperCase()})</script>
```

```
<template>
<div class="layout"> <div class="sidebar">
<div class="logo"> <h1>ТРАНС<span>ЛОГ</span></h1>
<p>ГРУЗОПЕРЕВОЗКИ</p> </div> <div class="user">
<div class="avatar">{{ initials }}</div> <div> <p class="user-name"> {{ fullname }}</p>
<p class="user-role">Менеджер</p> </div> </div> <div class="menu"> <div class="podmenu">

<a class="menu-item active" @click="$router.push('/addorder')">Заявки</a> </div>
<div class="podmenu_active"> 
<a class="menu-item" @click="$router.push('/trips')">Рейсы</a> </div> <div class="podmenu">

<a class="menu-item" @click="router.push('/drivers')">Водители</a>
</div> </div> <hr> <a class="logout" @click="logout">Выйти из системы</a> </div>
<div class="content"> <div class="topbar">Список рейсов</div>
```

Продолжение приложения А

```
<div class="card"> <h2>Список рейсов</h2> <div style="overflow-y: auto; max-height: 650px;">
<table> <thead> <tr>
<td>№ ЗАЯВКИ</td> <td>Водитель</td>
<td>Дата</td> <td>МАРШРУТ</td><td>Отправление</td> <td>Прибытие</td> <td>КМ</td> <td>ГРУЗ
(T)</td> <td>СТАТУС</td>
</tr> </thead><tbody>
<tr v-for="order in orders" :key="order.orderId">
<td>#{{ order.orderId }}</td> <td>{{ getDriverName(order.vehicleId) }}</td> <td>{{ new
Date(order.receivedAt).toLocaleDateString('ru-RU') }}</td> <td>{{ order.departurePoint }} →
{{ order.arrivalPoint }}</td> <td>{{ order.departureTime || '—' }}</td> <td>{{ order.arrivalTime || '—' }}</td>
<td>{{ order.distanceKm || '—' }}</td> <td>{{ order.weight }}</td> <td> <span
class="status":class="{
waiting: order.status === 'Ожидание', accepted: order.status === 'Принято',
paying: order.status === 'Ожидает оплаты', progress: order.status === 'Выполняется',
done: order.status === 'Доставлено', cancel: order.status === 'Отменено'}">{{ order.status }}</span>
</td></tr></tbody> </table> </div> </div> </div></div> </template>
```

spavki.vue

<script setup>

```
import router from '@router';
import logo from '../assets/images/logo.png'
import { ref, computed, onMounted } from 'vue'
import people from '../assets/images/users.png'
import spravka from '../assets/images/spavka.png'
import transport from '../assets/images/transport.png'
const showkmmodal = ref(false)
const selectedType = ref(null)
const vehiclesType = ref([])
const roles = ref([])
const fullname = localStorage.getItem('fullname') onMounted(async () => {
const response = await fetch('http://localhost:5095/api/VehicleType/GetTypes')
const data = await response.json()
vehiclesType.value = data
const response2 = await fetch('http://localhost:5095/api/Role/getRole')
const data2 = await response2.json()
roles.value = data2 })
const openTypeModal = (vehicle) =>

{ selectedType.value = vehicle
showkmmodal.value = true }
const errorMessageModal = ref("")
const saveType = async () =>
{ if (!selectedType.value.pricePerKm)
{ errorMessageModal.value = 'Заполните поле!' setTimeout(() => { errorMessageModal.value = "" }, 5000) return }
if (selectedType.value.pricePerKm > 9999)
{ errorMessageModal.value = 'Слишком большая цена!'
setTimeout(() => { errorMessageModal.value = "" }, 5000) return }
const response = await fetch(
'http://localhost:5095/api/VehicleType/UpdateType', {
```


Продолжение приложения А

```
method: "PUT", headers: { 'Content-Type': 'application/json' },
body: JSON.stringify( {
  "typeId": selectedType.value.typeId,
  "name": selectedType.value.name,
  "description": selectedType.value.description,
  "pricePerKm": selectedType.value.pricePerKm })})
showkmmodal.value = false
const response1 = await fetch('http://localhost:5095/api/VehicleType/GetTypes`)
vehiclesType.value = await response1.json()
const logout = () => { localStorage.clear()
router.push('/authorization')}
const initials = computed(() => {
if (!fullname) return "
return fullname.split(' ').map(word => word[0]).join("").toUpperCase() })
</script>
<template>
<div class="layout"> <div class="sidebar"> <div class="logo">
<h1>ТРАНС<span>ЛОГ</span></h1>
<p>ГРУЗОПЕРЕВОЗКИ</p> </div>
<div class="user"> <div class="avatar">{{ initials }}</div> <div>
<p class="user-name">{{ fullname }}</p> <p class="user-role">Администратор</p> </div> </div>
<div class="menu"> <div class="podmenu" @click="$router.push('/usersList')"> 
<a class="menu-item" > Пользователи</a> </div>
<div class="podmenu_active" >  <a class="menu-item" >Справочники</a> </div>
<div class="podmenu" @click="$router.push('/transport')"> 
<a class="menu-item">Транспорт</a> </div> </div> <hr>
<a class="logout" @click="logout">Выйти из системы</a> </div>
<div class="content"> <div class="topbar">Справочники</div>
<div class="cards" style="display: flex; flex-direction: row; justify-content: center; align-items: center;">
<div class="typeauto" > <h2>Типы автомобилей</h2>
<div class="title_row" v-for="v in vehiclesType" :key="v.typeId">
<div class="title_type"> <p style="font-weight: bold; font-size: 22px;">{{ v.name }}</p>
<p style="font-size: 16px; text-wrap: nowrap;">Стоимость за 1 км пути: {{ v.pricePerKm }}</p> </div>
<button class="simple-save" @click="openTypeModal(v)" style="width: 200px; margin-left: 170px; margin-top:
20px; font-size: 18px;">Изменить</button>
</div> </div> <div class="typeauto">
<h2>Статусы заявок</h2> <div class="statuses" style="display: flex; flex-direction: row; font-size: 20px; gap:
30px;" > <div style="display: flex; flex-direction: column; gap: 10px;">
<p style="background-color: gray;">Ожидание</p>
<p style="background-color: #9b59b6;">Ожидает оплаты</p>
<p style="background-color: #3498db;">Принято</p> </div>
<div style="display: flex; flex-direction: column; gap: 10px;">
<p style="background-color: orange;">Выполняется</p> <p style="background-color: green;">Доставлено</p>
<p style="background-color: red;">Отменено</p>
</div> </div> </div> </div>
<div class="typeauto" style="margin-left: 70px; height: 320px;"> <h2>Роли пользователей</h2>
<div class="title_row" v-for="r in roles" :key="r.roleId">
<div class="title_type"> <p style="font-weight: bold; margin-top: 20px; font-size: 22px;">{{ r.roleName }}</p>
</div> </div> </div> </div> </div> <div v-if="showkmmodal" class="simple-modal">
<div class="simple-modal-content"> <h2>Изменить цену за 1 км</h2>
```

Продолжение приложения А

```
<div class="simple-field"> <label>Цена за 1 км</label>
<input v-model.number="selectedType.pricePerKm" type="number" min="0" max="9999">
</div> <div class="simple-buttons">
<button class="simple-cancel" @click="showkmmodal = false">Отмена</button>
<button class="simple-save" @click="saveType">Сохранить</button>
</div> <p v-if="errorMessageModal" style="display:flex; justify-content: center; align-items: center; color: red;
font-size: 14px; margin-top: 5px;">{{ errorMessageModal }}</p> </div> </div>
</template>
```

transport.vue

```
<script setup>
```

```
import router from '@router';
import logo from '../assets/images/logo.png'
import { ref, computed, onMounted } from 'vue'
import people from '../assets/images/users.png'
import spravka from '../assets/images/spavka.png'
import transport from '../assets/images/transport.png'
const showTripModal = ref(false)
const transports = ref([])
const vehicleTypes = ref([])
const drivers = ref([])
const fullname = localStorage.getItem('fullname')
const loadTransports = async () => { const r = await fetch(`http://localhost:5095/api/Vehicle/GetTransport`)
transports.value = await r.json() }
const freeDrivers = ref([])
onMounted(async () => { await loadTransports()
const r2 = await fetch(`http://localhost:5095/api/VehicleType/GetTypes`)
vehicleTypes.value = await r2.json()
const r3 = await fetch(`http://localhost:5095/api/User/GetUser`)
const users = await r3.json()
const r4 = await fetch(`http://localhost:5095/api/Driver/GetDrivers`) const allDrivers = await r4.json()
const busyUserIds = new Set(allDrivers.filter(d => d.vehicleId).map(d => d.userId))
freeDrivers.value = users.filter(u => u.roleId === 3 && !busyUserIds.has(u.userId)) })
const logout = () => {
localStorage.clear()
router.push('/authorization') }
const showAddTransport = ref(false)
const selectedTransport = ref(null)
const opentransportmodal = (vehicle) => {
selectedTransport.value = vehicle
showAddTransport.value = true }
const newTransport = ref({ licensePlate: "", brand: "", model: "", payloadKg: null, volumeM3: null, vehicleTypeId:
null, userId: null })
const errorModalmessage = ref("")
const message = ref("")
const addTransport = async () => {
if (!newTransport.value.licensePlate || !newTransport.value.brand || !newTransport.value.model || !
newTransport.value.payloadKg || !newTransport.value.volumeM3) { errorModalmessage.value = 'Заполните поля!'

```

Продолжение приложения А

```
setTimeout(() => { errorMessage.value = " ", 5000) return }
if (!newTransport.value.vehicleTypeId)
{ errorMessage.value = 'Выберите тип транспорта!'
setTimeout(() => { errorMessage.value = " ", 5000) return}
await fetch(`http://localhost:5095/api/Vehicle/AddTransport`, { method: 'POST', headers: { 'Content-Type':
'application/json' },
body: JSON.stringify(newTransport.value) }) errorMessage.value = "message.value = 'Транспорт успешно
добавлен!' setTimeout(() => { message.value = " ", 5000) newTransport.value = { licensePlate: " ", brand: " ", model:
" ", payloadKg: null, volumeM3: null, vehicleTypeId: null, userId: null } showAddTransport.value = false await
loadTransports() await reloadFreeDrivers() }
const showEditTransport = ref(false)
const editTransport = ref(null)
const openEditTransport = (vehicle) =>
{ editTransport.value = { vehicleId: vehicle.vehicleId, licensePlate: vehicle.licensePlate,
brand: vehicle.brand, model: vehicle.model, payloadKg: vehicle.payloadKg,
volumeM3: vehicle.volumeM3, vehicleTypeId: vehicle.vehicleTypeId,
userId: vehicle.drivers?.[0]?.userId ?? null } showEditTransport.value = true }
const saveTransport = async () =>
{ if (!editTransport.value.licensePlate || !editTransport.value.brand || !editTransport.value.model || !
editTransport.value.payloadKg || !editTransport.value.volumeM3 ){
errorMessage.value = 'Заполните поля!'
setTimeout(() => { errorMessage.value = " ", 5000)
return } if (!editTransport.value.vehicleTypeId) {
errorMessage.value = 'Выберите тип транспорта!'
setTimeout(() => { errorMessage.value = " ", 5000)
return }
await fetch(`http://localhost:5095/api/Vehicle/UpdateTransport?vehicleId=${editTransport.value.vehicleId}`, {
method: 'PUT', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(editTransport.value) })
errorMessage.value = "message.value = 'Транспорт успешно изменен!' setTimeout(() => { message.value =
" ", 5000)
showEditTransport.value = false
await loadTransports() await reloadFreeDrivers() }
const initials = computed(() => { if (!fullname) return "
return fullname.split(' ').map(word => word[0]).join("").toUpperCase() })
const deleteTransport = async (vehicle) => {
await fetch(`http://localhost:5095/api/Vehicle/DeleteTransport?vehicleId=${vehicle.vehicleId}`,
{ method: 'DELETE' })
message.value = "Транспорт успешно удален!"
setTimeout(() => { message.value = " ", 5000)
await loadTransports() }
const editDrivers = computed(() => {
if (!editTransport.value) return freeDrivers.value
const currentDriver = transports.value
.find(t => t.vehicleId === editTransport.value.vehicleId)
?.drivers?.[0]?.user
if (!currentDriver) return freeDrivers.value
const alreadyIn = freeDrivers.value.some(d => d.userId === currentDriver.userId)
return alreadyIn ? freeDrivers.value : [...freeDrivers.value, currentDriver] })
const reloadFreeDrivers = async () => {
const r3 = await fetch(`http://localhost:5095/api/User/GetUser`)
```

Продолжение приложения А

```
const users = await r3.json()
const r4 = await fetch('http://localhost:5095/api/Driver/GetDrivers')
const allDrivers = await r4.json()
const busyUserIds = new Set(allDrivers.filter(d => d.vehicleId).map(d => d.userId))
freeDrivers.value = users.filter(u => u.roleId === 3 && !busyUserIds.has(u.userId)) }
</script>
<template>
<div class="layout">
<div class="sidebar"> <div class="logo"> <h1>ТРАНСПОРТ</h1> <p>ГРУЗОПЕРЕВОЗКИ</p>
</div> <div class="user"> <div class="avatar">{{ initials }}</div> <div>
<p class="user-name">{{ fullname }}</p> <p class="user-role">Администратор</p> </div> <div
class="menu">
<div class="podmenu" @click="$router.push('/usersList')"> 
<a class="menu-item"> Пользователи</a> </div>
<div class="podmenu" @click="$router.push('/spavki')"> 
<a class="menu-item">Справочники</a> </div>
<div class="podmenu_active">  <a class="menu-item">Транспорт</a>
</div> </div> <hr> <a class="logout" @click="logout">Выйти из системы</a> </div>
<div class="content"> <div class="topbar">Транспорт</div> <div class="card"> <div class="title_card">
<h2 id="titleorder" style="margin-top: -30px;">Транспортные средства</h2>
<button class="simple-save" style="height: 40px; width: 200px; font-size: 18px;" @click="showAddTransport =
true">Добавить</button> </div>
<div style="overflow-y: auto; max-height: 680px; text-wrap: nowrap;">
<table> <thead> <tr> <td>Гос. Номер</td> <td>Марка</td>
<td>Грузоподъёмность (кг)</td> <td>Объём (м³)</td> <td>Тип</td> <td>Водитель</td> </tr>
</thead> <tbody> <tr v-for="t in transports" :key="t.vehicleId"> <td>{{ t.licensePlate }}</td>
<td>{{ t.brand }} {{ t.model }}</td> <td>{{ t.payloadKg }}</td>
<td>{{ t.volumeM3 }}</td> <td>{{ t.vehicleType?.name ?? 'Не указан' }}</td>
<td>{{ t.drivers?.[0]?.user?.fullName ?? 'Не назначен' }}</td> <td>
<div style="display: flex; flex-direction: row; gap: 5px;">
<button class="manbtn" @click="openEditTransport(t)">Изменить</button>
<button class="manbtn" @click="deleteTransport(t)" style="background-color: red;">Удалить</button>
</div> </td> </tr> </tbody> </table>
</div> <p style="color: green; margin-top: 20px; font-size: 16px; font-weight: bold; padding-left: 15px;" v-
if="message"> {{ message }} </p> </div> </div> </div>
<div v-if="showAddTransport" class="simple-modal">
<div class="simple-modal-content"> <h2>Добавить транспорт</h2>
<div class="simple-row"> <div class="simple-field">
<label>Гос. номер</label>
<input v-model="newTransport.licensePlate" placeholder="A001AA 777"> </div>
<div class="simple-field">
<label>Тип</label> <select v-model="newTransport.vehicleTypeId" class="simple-select">
<option :value="null">Выберите тип</option>
<option v-for="vt in vehicleTypes" :key="vt.typeId" :value="vt.typeId">{{ vt.name }}</option>
</select> </div> </div>
<div class="simple-row"> <div class="simple-field">
<label>Марка</label>
<input v-model="newTransport.brand" placeholder="КАМАЗ">
</div> <div class="simple-field">
<label>Модель</label>
```

Продолжение приложения А

```
<input v-model="newTransport.model" placeholder="5490"> </div> </div>
<div class="simple-row"> <div class="simple-field"> <label>Грузоподъёмность (кг)</label>
<input type="number" v-model="newTransport.payloadKg">
</div> <div class="simple-field"> <label>Объём (м³)</label>
<input type="number" v-model="newTransport.volumeM3"> </div> </div>
<div class="simple-field">
<label>Водитель</label>
<select v-model="newTransport.userId" class="simple-select">
<option :value="null">Без водителя</option>
<option v-for="d in freeDrivers" :key="d.userId" :value="d.userId">{{ d.fullName }}</option> </select> </div>
<div class="simple-buttons"> <button class="simple-cancel" @click="showAddTransport =
false">Отмена</button>
<button class="simple-save" @click="addTransport">Сохранить</button> </div>
<p style="color: red; font-size: 14px; display: flex; align-items: center; justify-content: center; margin-top: 5px" v-
if="errorModalmessage" > {{ errorModalmessage }}</p> </div> </div>
<div v-if="showEditTransport" class="simple-modal">
<div class="simple-modal-content"> <h2>Изменить транспорт</h2>

<div class="simple-row"> <div class="simple-field"> <label>Гос. номер</label>
<input v-model="editTransport.licensePlate"> </div>
<div class="simple-field"> <label>Тип</label>
<select v-model="editTransport.vehicleTypeId" class="simple-select">
<option :value="null">Выберите тип</option>
<option v-for="vt in vehicleTypes" :key="vt.typeId" :value="vt.typeId">{{ vt.name }}</option>
</select> </div> </div>
<div class="simple-row"><div class="simple-field">
<label>Марка</label>
<input v-model="editTransport.brand"> </div>
<div class="simple-field"> <label>Модель</label>
<input v-model="editTransport.model"> </div></div>
<div class="simple-row"> <div class="simple-field">
<label>Грузоподъёмность (кг)</label>
<input type="number" v-model="editTransport.payloadKg"> </div>
<div class="simple-field">
<label>Объём (м³)</label>
<input type="number" v-model="editTransport.volumeM3"> </div> </div>
<div class="simple-field">
<label>Водитель</label>
<select v-model="editTransport.userId" class="simple-select">
<option :value="null">Без водителя</option>
<option v-for="d in editDrivers" :key="d.userId" :value="d.userId">{{ d.fullName }}</option> </select> </div>
<div class="simple-buttons">
<button class="simple-cancel" @click="showEditTransport = false">Отмена</button>
<button class="simple-save" @click="saveTransport">Сохранить</button> </div>
<p style="color: red; font-size: 14px; display: flex; align-items: center; justify-content: center; margin-top: 5px" v-
if="errorModalmessage" > {{ errorModalmessage }}</p> </div> </div>
</template>
usersList.vue

<script setup>
```

Продолжение приложения А

```
import router from '@router';
import logo from '../assets/images/logo.png'
import { ref, computed, onMounted } from 'vue'
import people from '../assets/images/users.png'
import spravka from '../assets/images/spavka.png'
import transport from '../assets/images/transport.png'
const showUsermodal = ref(false)
const selectedUser = ref(null)
const users = ref([])
const role = ref("")
const message = ref("")
const fullname = localStorage.getItem('fullname')
const errorModalmessage = ref("")
const loadUsers = async () => { const response = await fetch(`http://localhost:5095/api/User/GetUser`)
const data = await response.json() users.value = data }
onMounted(loadUsers) const getRoleName = (roleId) => { switch (roleId) {
case 1: return 'Администратор' case 2: return 'Менеджер' case 3: return 'Водитель' case 4: return 'Клиент'
default: return 'Неизвестно'}} const openUserModal = (user) => {
selectedUser.value = user showUsermodal.value = true } const saveUser = async () => {
if (!selectedUser.value.fullName || !selectedUser.value.username || !selectedUser.value.numberphone) {
errorModalmessage.value = 'Заполните все поля!' setTimeout(() => { errorModalmessage.value = "" }, 5000)
return }
const digits = selectedUser.value.numberphone.replace(/D/g, "") if (digits.length !== 11) {
errorModalmessage.value = 'Введите корректный номер телефона!' setTimeout(() => { errorModalmessage.value
= "" }, 5000) return } const response = await fetch(`http://localhost:5095/api/User/UpdateUser`, {
method: "PUT", headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({
userId: selectedUser.value.userId, fullName: selectedUser.value.fullName,
username: selectedUser.value.username, numberphone: selectedUser.value.numberphone,
roleId: selectedUser.value.roleId })))
showUsermodal.value = false await loadUsers() }
const deleteuser = async (user) => { const response = await fetch(
`http://localhost:5095/api/User/DeleteUser?userId=${user.userId}`, { method: "DELETE" })
if (response.ok) { message.value = 'Пользователь удалён'
setTimeout(() => { message.value = "" }, 5000)}
await loadUsers() }
const logout = () => { localStorage.clear() router.push('/authorization') }
const showAddUser = ref(false)
const newUser = ref({ fullName: "", username: "", numberphone: "", roleId: 4, password: "" })
const openShowAddUser = (user) => {
showAddUser.value = true }const Adduser = async () => {
if (!newUser.value.fullName || !newUser.value.username || !newUser.value.password || !
newUser.value.numberphone) { errorModalmessage.value = 'Заполните все поля!'
setTimeout(() => { errorModalmessage.value = "" }, 5000) return }
const digits = newUser.value.numberphone.replace(/D/g, "") if (digits.length !== 11) {
errorModalmessage.value = 'Введите корректный номер телефона!'
setTimeout(() => { errorModalmessage.value = "" }, 5000) return }
const response = await fetch(`http://localhost:5095/api/User/register`, {
method: 'POST', headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({username: newUser.value.username,
password: newUser.value.password, roleId: newUser.value.roleId,
```

Продолжение приложения А

```
fullname: newUser.value.fullName, numberphone: newUser.value.numberphone,
isActive: 1 }) })
const data = await response.json() if (response.ok) { await loadUsers()
newUser.value = { fullName: "", username: "", numberphone: "", roleId: 4, password: "" } message.value = data.message
setTimeout(() => { message.value = "" }, 5000) showAddUser.value = false } else {
message.value = data.message setTimeout(() => { message.value = "" }, 5000) })
const initials = computed(() => { if (!fullname) return ""
return fullname.split(' ').map(word => word[0]).join('').toUpperCase() })
const handlePhoneFocus = (e) => { if (!e.target.value) e.target.value = '+7'
setTimeout(() => e.target.setSelectionRange(e.target.value.length, e.target.value.length), 0) }
const handlePhoneInput = (e) => { let val = e.target.value
if (!val.startsWith('+7')) val = '+7' val = '+7' + val.slice(2).replace(/D/g, "")
if (val.length > 12) val = val.slice(0, 12) if (showUsermodal.value) selectedUser.value.numberphone = val
else newUser.value.numberphone = val }
</script>
<template>
<div class="layout"> <div class="sidebar"> <div class="logo">
<h1>ТРАНС<span>ЛОГ</span></h1> <p>ГРУЗОПЕРЕВОЗКИ</p> </div>
<div class="user"> <div class="avatar">{{ initials }}</div>
<div> <p class="user-name">{{ fullname }}</p>
<p class="user-role">Администратор</p> </div> </div>
<div class="menu"> <div class="podmenu_active"> 
<a class="menu-item" > Пользователи</a> </div>
<div class="podmenu" @click="$router.push('/spavki')" > 
<a class="menu-item" >Справочники</a> </div>
<div class="podmenu" @click="$router.push('/transport')" > 
<a class="menu-item">Транспорт</a> </div> </div> <hr>
<a class="logout" @click="logout">Выйти из системы</a> </div> <div class="content">
<div class="topbar">Пользователи</div>
<div class="card" > <div class="title_card">
<h3 id="titleorder" style="margin-top: -30px;">Все пользователи</h3>
<button class="simple-save" style="height: 40px; width: 200px; font-size: 18px;"
@click="openShowAddUser">Добавить</button> </div>
<div style="overflow-y: auto; max-height: 610px; width: 100%;" > <table> <thead> <tr>
<td>ФИО</td> <td>Логин</td> <td>Номер телефона</td> <td>Роль</td> </tr> </thead> <tbody>
<tr v-for="u in users" :key="u.userId">
<td>{{ u.fullName }}</td> <td>{{ u.username }}</td> <td>{{ u.numberphone }}</td>
<td>{{ getRoleName(u.roleId) }}</td> <td>
<div style="display: flex; flex-direction: row; gap: 5px;">
<button class="manbtn" @click="openUserModal(u)">Изменить</button>
<button class="manbtn" @click="deleteuser(u)" style="background-color: red;">Удалить</button>
</div> </td> </tr> </tbody> </table> </div>
<p style="color: green; margin-top: 20px; font-size: 16px; font-weight: bold; padding-left: 15px;"
v-if="message">{{ message }} </p>
</div> </div> </div>
<div v-if="showUsermodal" class="simple-modal">
<div class="simple-modal-content">
<h2>Изменить пользователя</h2>
<div class="simple-field">
<label>ФИО</label>
```

Продолжение приложения А

```
<input v-model="selectedUser.fullName"> </div>
<div class="simple-field">
<label>Логин</label>
<input v-model="selectedUser.username"> </div>
<div class="simple-field">
<label>Номер телефона</label>
<input v-model="selectedUser.numberphone" placeholder="+7 (900) 321-67-52" maxlength="12"
@focus="handlePhoneFocus" @input="handlePhoneInput"> </div>
<div class="simple-field"> <label>Роль</label>
<select v-model="selectedUser.roleId" class="simple-select">
<option :value="1">Администратор</option>
<option :value="2">Менеджер</option>
<option :value="3">Водитель</option>
<option :value="4">Клиент</option> </select> </div>
<div class="simple-buttons">
<button class="simple-cancel" @click="showUsermodal = false">Отмена</button>
<button class="simple-save" @click="saveUser">Сохранить</button> </div>
<p style="color: red; font-size: 14px; display: flex; align-items: center; justify-content: center; margin-top: 5px" v-
if="errorModalmessage" > {{ errorModalmessage }}</p> </div> </div>
<div v-if="showAddUser" class="simple-modal"> <div class="simple-modal-content">
<h2>Добавление пользователя</h2>
<div class="simple-row">
<div class="simple-field">
<label>ФИО</label>
<input placeholder="Введите ФИО" v-model="newUser.fullName"> </div>
<div class="simple-field">
<label>Логин</label>
<input placeholder="Введите логин" v-model="newUser.username"> </div> </div>
<div class="simple-row">
<div class="simple-field">
<label>Номер телефона</label>
<input v-model="newUser.numberphone" placeholder="+7 (900) 321-67-52" maxlength="12"
@focus="handlePhoneFocus" @input="handlePhoneInput"> </div>
<div class="simple-field">
<label>Роль</label>
<select class="simple-select" v-model="newUser.roleId">
<option value="Выберите роль">Выберите роль</option>
<option :value="1">Администратор</option> <option :value="2">Менеджер</option>
<option :value="3">Водитель</option> <option :value="4">Клиент</option>
</select> </div> </div>
<div class="simple-field">
<label>Пароль</label>
<input v-model="newUser.password" type="password"> </div>
<div class="simple-buttons">
<button class="simple-cancel" @click="showAddUser = false">Отмена</button>
<button class="simple-save" @click="Adduser">Сохранить</button> </div>
<p style="color: red; font-size: 14px; display: flex; align-items: center; justify-content: center; margin-top: 5px" v-
if="errorModalmessage" > {{ errorModalmessage }}</p> </div> </div>
</template>
```